MALLA REDDY UNIVERSITY / MALLA REDDY VISHWAVIDHYAPEETH

Hyderabad, Telangana

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

HabitFlow

HABIT TRACKING APPLICATION

CLASS LAB EXTERNAL HACKATHON PROJECT REPORT

Problem Statement: Habit Tracking Application

TEAM NUMBER: 5

ROLE	NAME	STUDENT ID
Team Leader	Gandham Bharath	25MVCSMR0643
Team Member	A. Harish	25MVCSMR0633
Team Member	CH. Manipradeep	25MVCSMR0679
Team Member	K.HarshaVardhan	25MVCSMR0653

ACADEMIC YEAR: 2026–2027

B.Tech 2nd Year | CSE – Artificial Intelligence and Machine Learning

PAGE 2 | ABSTRACT & INTRODUCTION

Abstract

HabitFlow is a proposed MERN Stack web application for recording daily habits and reviewing progress in a structured way. It addresses the difficulty of maintaining consistent routines when people rely on memory, paper notes, or scattered reminders. The application is designed to let a user create habits, define targets or schedules, mark a habit as completed for a particular date, edit or remove records, and review completion history. MongoDB is intended to persist habit details and completion records; Express.js exposes REST API endpoints; Node.js runs the server-side application logic; and React.js provides the interactive user interface. Together, these components separate presentation, request handling, application logic, and data storage. The system can calculate daily and weekly completion rates from scheduled habit occurrences and present the results as summaries or visual indicators. Where authentication is implemented, each habit should be associated with its owner and protected by authorization checks. The report describes the project requirements, architecture, data model, API design, implementation approach, testing plan, and possible extensions. As implementation evidence and execution logs were not supplied for this report, proposed functions and test outcomes are distinguished from verified results. HabitFlow is intended to help students and other users build consistency, reflect on routines, and make progress visible without making unsupported claims about productivity outcomes.

1. Introduction

1.1 Background and Importance

Habits are repeated actions that support learning, health, organization, and personal development. Tracking them makes goals more observable: a user can compare intended activity with recorded completion and identify patterns over time. A digital tracker can retain history, reduce manual calculation, and present progress in a consistent format.

1.2 Existing Problem and Motivation

Manual habit logs can be misplaced, inconsistently updated, or difficult to summarize. Separate notes and reminder applications may not provide a single view of targets, completion status, and historical progress. HabitFlow is motivated by the need for a simple, centralized interface in which habit records can be created, updated, reviewed, and evaluated.

1.3 Proposed Solution

HabitFlow organizes each habit around its name, schedule or target, and dated completion records. Users interact with a React dashboard, while REST requests pass through an Express API to server-side logic running on Node.js. MongoDB stores the persistent records. The intended result is a maintainable web application that makes daily tracking and progress review straightforward.

PAGE 3 | PROBLEM STATEMENT, OBJECTIVES & SCOPE

2. Official Problem Statement

Build a MERN-based habit tracking application that tracks daily habits and progress. The system should manage habit names, dates, targets, and completion status using MongoDB. React.js should provide the user interface, while Node.js and Express.js should handle backend APIs. The application should support the main operations required for the project, including adding, viewing, updating, and managing relevant records. The project should demonstrate MERN Stack development, implement CRUD operations using MongoDB, build REST APIs using Node.js and Express.js, develop the frontend using React.js, and implement authentication and role-based access where required.

3. Project Objectives

1. Develop a user-friendly daily habit tracking application.
2. Implement Create, Read, Update, and Delete (CRUD) operations for habit records.
3. Store and manage habit information using MongoDB.
4. Develop RESTful APIs using Node.js and Express.js.
5. Create an interactive frontend using React.js.
6. Track daily completion status, targets, and progress.
7. Display habit statistics and progress summaries.
8. Implement authentication and access control where appropriate.

4. Project Scope

4.1 Core Scope

- Create, view, update, and delete personal habit records.
- Record whether a habit was completed on a specific date.
- Store targets and schedules and calculate progress from available records.
- Review dated history and daily or weekly summaries.

4.2 Proposed / Conditional Scope

- Visual charts, calendar views, streaks, and reminders, if not present in the current build.
- User registration, authentication, and owner-specific access controls, if not yet implemented.
- Deployment, mobile-focused refinements, and expanded analytics.

Implementation note: The scope above describes the intended system. Features should be reported as implemented only after they are confirmed in the working build. No source code, screenshots, or execution logs were supplied with the request.

PAGE 4 | FEATURES & FUNCTIONAL REQUIREMENTS

5. Functional Requirements

ID	Function	Requirement / Expected Behaviour
FR-01	Create habit	Accept a habit name and relevant schedule/target details; validate and save the record.
FR-02	View habits	Retrieve and display saved habits and their current status.
FR-03	Update habit	Edit an existing habit's name, target, schedule, or other permitted fields.
FR-04	Delete habit	Remove a selected habit after confirming the intended action.
FR-05	Record completion	Set completion status for a specific habit and date.
FR-06	Calculate progress	Compute completion summaries from scheduled occurrences and recorded status.
FR-07	View history	Show dated completion records and available daily or historical summaries.
FR-08	Access control	If authentication is implemented, restrict records to the authorized owner and enforce roles where needed.

6. Non-Functional Requirements

Quality Attribute	Requirement
Usability	Clear labels, understandable status indicators, and a simple workflow.
Performance	Use appropriately filtered queries and avoid unnecessary API requests; measure performance during testing.
Reliability	Handle invalid requests and database failures without silently losing data.
Data consistency	Validate required fields and use a consistent date format and completion-record structure.
Maintainability	Separate React components, API routes, business logic, and database access.
Security	Validate inputs, protect secrets, use authorization checks, and avoid exposing sensitive data.
Responsive design	Support common desktop, tablet, and mobile viewport sizes; verify through device testing.

7. User Workflow and Component Responsibilities

Component	Responsibility
User	Creates or edits habits and marks completion.
React.js frontend	Displays forms, habit lists, status controls, and summaries; sends HTTP requests.
Express.js REST API	Maps routes to handlers, validates request shape, and returns HTTP status codes with JSON.
Node.js backend	Runs application logic, applies business rules, and coordinates database operations.
MongoDB	Persists habit documents and dated completion records; returns query or write results.

Workflow: User → React Frontend → Express REST API → Node.js Backend → MongoDB Database → API Response → Frontend

PAGE 5 | TECHNOLOGIES, ARCHITECTURE, IMPLEMENTATION & TESTING

8. MERN Stack and System Architecture

Technology	Role in HabitFlow
MongoDB	Document database for habit details, targets, schedules, completion records, and optional user references.
Express.js	Backend web framework for defining REST endpoints, middleware, validation, and error responses.
React.js	Component-based frontend for the dashboard, habit form, habit list, completion controls, and progress view.
Node.js	JavaScript runtime for server execution and application logic; coordinates API handling and persistence.

8.1 Text-Based Architecture

```
User
  ↓ interacts with
React.js Frontend (Dashboard, Habit Form, Habit List, Progress Summary)
  ↓ HTTP/JSON requests and responses
Express.js REST API (Routes, Validation, Error Responses)
  ↓ invokes
Node.js Application Logic / Controllers
  ↓ reads and writes through database driver/ODM
MongoDB Database (Habits, Completion Records, Users if authentication is enabled)
  ↑ result returned through API → React UI updates
```

9. Proposed Data Model

```
Illustrative habit document (field names may be adapted to the actual schema):
{
  "name": "Read for 20 minutes",
  "target": { "value": 20, "unit": "minutes" },
  "schedule": { "daysOfWeek": [1, 2, 3, 4, 5] },
  "completionRecords": [
    { "date": "2026-10-10", "completed": true }
  ],
  "userId": "example-user-id"
}
The userId field is applicable only when user accounts and ownership are implemented. Dates should use a consistent, documented
convention; a date-only value can avoid unintended time-zone shifts for daily records.
```

10. Example REST API Endpoints

Method	Endpoint	Purpose
POST	/api/habits	Create a habit after validation.
GET	/api/habits	List habits available to the current user or project context.
GET	/api/habits/:id	Retrieve one habit by identifier.
PUT/PATCH	/api/habits/:id	Update permitted habit fields or schedule.
DELETE	/api/habits/:id	Delete a habit, with ownership checks where applicable.
PUT/PATCH	/api/habits/:id/completions/:date	Create or update completion status for a date.

11. Implementation and Testing Approach

Implementation should separate UI components, API routes/controllers, validation, and database access. The frontend sends JSON requests; the backend validates required fields and identifiers, applies date and ownership rules where applicable, performs MongoDB operations, and returns an appropriate status code. Errors should be handled centrally and shown to users without exposing stack traces or secrets. Test the happy path, invalid inputs, duplicate completion dates, missing records, database failures, and authorization boundaries. The following test cases are a plan, not completed results.

Test Case	Input / Action	Expected Result	Status
Create valid habit	Submit a name and valid target	Habit is saved and appears in the list	To be verified
Reject invalid input	Submit a blank name or invalid target	API returns a validation error; no invalid record is stored	To be verified
Read habits	Open dashboard / call GET endpoint	Saved habits are returned and displayed	To be verified
Update habit	Change a target or schedule	Updated values persist after reloading	To be verified
Delete habit	Delete a selected record	Record is removed or marked deleted according to design	To be verified
Mark completion	Mark a habit complete for a date	Correct date/status is stored and reflected in UI	To be verified

Duplicate date entry	Submit completion twice for same habit/date	System updates the existing date entry or rejects duplicates consistently	To be verified
No scheduled habits	Request daily progress with denominator zero	Display a defined empty state; do not divide by zero	To be verified
Unauthorized access	Attempt to access another user's record, if accounts exist	Request is denied without leaking record data	To be verified
Database/API failure	Simulate unavailable database or server error	User receives a safe error message; server logs diagnostic details	To be verified

Results status: Actual test execution was not evidenced in the materials provided. All cases remain "To be verified"; no performance figures or deployment claims are made.

PAGE 6 | PROGRESS LOGIC, CONCLUSION & FUTURE SCOPE

12. Habit Progress Calculation

Progress should be calculated from habits scheduled for the selected period, rather than treating every saved habit as due every day. For a selected date, let C be the number of scheduled habit occurrences marked complete and S be the total number of occurrences scheduled for that date.

Daily completion rate = (C / S) × 100, when S > 0

For a week, let CW be completed scheduled occurrences and SW be all scheduled occurrences in the week. Weekly completion rate = (CW / SW) × 100, when SW > 0. If the denominator is zero, the application should show "No habits scheduled" or an equivalent empty state, not divide by zero. Completion records should be matched to a consistent date convention, and repeated updates for the same habit/date should not inflate the count.

13. Conclusion

HabitFlow translates the habit-tracking problem into a practical full-stack application design. Its intended CRUD workflow, dated completion records, and progress calculations provide a clear use case for connecting a React interface to Express REST APIs, Node.js business logic, and MongoDB persistence. The project offers second-year students practical experience in API design, data modelling, validation, error handling, and testing. Final evaluation should be based on a running build and evidence that each requirement passes its test case.

14. Limitations

Current implementation status cannot be confirmed from the supplied project description alone. Authentication, role-based access, charts, reminders, deployment, and mobile responsiveness must therefore be treated as conditional or future work until demonstrated. Progress summaries are only as reliable as the schedules and completion records entered by users.

15. Future Enhancements

Add secure registration and login; enforce user-level ownership; introduce streak tracking and configurable reminders; provide calendar views, weekly analytics, and charts; improve accessibility and mobile layouts; add export/backup options; and deploy with environment-based configuration, HTTPS, monitoring, and database backups.

16. References

9. React documentation — Learn React. https://react.dev/learn
10. Node.js documentation. https://nodejs.org/docs/latest/api/
11. Express.js documentation. https://expressjs.com/
12. MongoDB Manual. https://www.mongodb.com/docs/manual/

THANK YOU

Team 5 | HabitFlow – Habit Tracking Application